

Brocamp - 1st Project Implementation Guidelines

1. Consistent Naming Conventions

- Use descriptive names for variables, functions, classes, and database entities that clearly reflect their purpose (e.g., `createUser`, `updatePost`, `deleteComment`).
- Follow consistent naming conventions: `camelCase` for variables and functions, `PascalCase` for classes.

2. Clear and User-Friendly Error Messaging

- Provide clear, concise, and helpful error messages during validation (e.g., *"Please enter a valid email address"*).
- Display errors near the relevant input fields so users can easily understand what needs to be corrected.

3. Confirm Before Deleting Data

- Always show a confirmation prompt before deleting any record to prevent accidental data loss.

4. Secure Authentication Practices

- Hash and salt passwords securely using `bcrypt`. Never store or log passwords in plain text.
- Enforce password strength rules: minimum length, uppercase, lowercase, digits, and special characters.
- Clearly communicate password rules to users when they are setting a password.

5. Proper Error Handling

- Use informative but secure error messages (e.g., *"Invalid username or password"*); avoid messages that reveal whether the username or password was wrong.
- Never expose sensitive system or database information through error responses.

- Display custom error pages for common errors (e.g., 404 Not Found, 500 Server Error).

6. File Upload Handling

- Restrict accepted file types to only what the project requires (e.g., images only).
- Limit file sizes to reasonable maximums for performance and security.
- Sanitize or generate unique filenames to prevent conflicts or security issues.
- Provide clear feedback to users on upload success or failure.

7. UI/UX Standards

- Maintain a consistent design style - colors, fonts, buttons, and spacing - across all pages.
- Ensure the application is fully responsive across various screen sizes using flexible layouts and media queries.
- Application UI should match the Figma design.
- Provide intuitive navigation with clear menus and links. Avoid broken or inactive links.
- Set the **cursor: pointer** on all clickable elements.
- Use **external CSS stylesheets** only - avoid inline or internal styles.
- If using a **frontend framework** (e.g., React, Angular, Vue): organize components logically, keep them reusable, and avoid unnecessary re-renders.
- If using a **template engine** (e.g., EJS, Handlebars, Pug): keep templates clean, avoid embedding business logic in views, and reuse partials where possible.

8. Admin Panel - Data Display

- Implement **pagination** to manage large data sets.
- Include **search and filtering** options for quick access to relevant records.
- Allow **sorting** by various fields, defaulting to descending order.
- Provide clear navigation controls for paging through data.

9. Folder Structure & Architecture

- Choose an appropriate architecture for your project based on your tech stack:
 - MERN / MEAN (JavaScript/Node.js) - MVC (Model-View-Controller)

- Python (Django/Flask) - MVT (Model-View-Template)
- .NET (C#) - MVC or Clean Architecture
- Ruby on Rails - MVC (Rails convention)
- Java (Spring Boot) - MVC or Layered Architecture (Controller-Service-Repository)
- Golang - MVC or simple layered structure (Handler-Service-Repository)
- **Organize your project files by resource** (e.g., `users/`, `products/`, `orders/`), keeping related routes, controllers, and models together under each resource.
- Keep config files (`.env`, `.gitignore`, `package.json`) properly placed.
- Avoid bloated or redundant files. Store static assets (images, fonts, icons) in a dedicated folder.
- Follow consistent naming conventions for all folders and files.

10. Code Quality & Best Practices

- Write modular, reusable code - avoid large monolithic files.
- Use environment variables and constants - no hardcoded values.
- Format code consistently using **Prettier** or **ESLint**.
- Write comments to explain complex or non-obvious logic.
- Audit dependencies for known vulnerabilities.

11. Security Best Practices

- No hardcoded credentials, API keys, or secrets - use `.env` files.
- Validate all user inputs to prevent **SQL Injection**, **XSS**, and **CSRF**.
- Implement proper **Role-Based Access Control (RBAC)**.
- Use **HTTPS** and apply secure headers (e.g., Helmet, CORS configuration).
- Use secure authentication with **JWT** or **OAuth**, including token expiration and refresh mechanisms.
- Learn about the **OWASP Top 10** vulnerabilities.

12. API Standards

- Follow **REST principles**: use correct HTTP methods (GET, POST, PUT, DELETE).
- Secure all endpoints with proper authentication and authorization.
- Return structured, consistent API responses with proper error handling.
- Implement pagination for endpoints that return large data sets.
- Follow **OpenAPI standards** and document your APIs using Swagger or Postman.

13. Database Best Practices

- Design an efficient schema suited to your database type:
 - **SQL** (MySQL, PostgreSQL) - proper normalization, well-defined relationships, foreign key constraints.
 - **NoSQL** (MongoDB) - thoughtful document structure, embed vs. reference decisions based on access patterns.
 - **Other** (Firebase, Redis, etc.) - follow the data modeling conventions recommended for that platform.
- Index frequently queried fields to improve query performance.
- Avoid N+1 queries; write efficient joins (SQL) or use proper aggregation pipelines (NoSQL).
- Enforce data integrity using constraints where your database supports it (foreign keys, unique indexes for SQL; schema validation for MongoDB).
- Never expose sensitive data through queries or API responses.

14. Logging

- Implement structured logging using a library such as **Winston**.
- Log meaningful application events and errors.
- Avoid excessive or unnecessary `console.log` statements - especially in production.

15. Git & Version Control

- Use **feature branches** for all new functionality - never commit directly to `main`.
- Write clear, meaningful commit messages with type prefixes (e.g., `feat:`, `fix:`, `chore:`).
- Push changes frequently for backup and collaboration.

- Use a `.gitignore` to exclude dependencies, environment configs, logs, and build artifacts.
- Never commit sensitive files (API keys, passwords, `.env` files) to the repository.

16. Deployment

- Deploy your project on a cloud provider - **AWS**, **Azure**, or **GCP**.
- Use environment-specific configuration files (`.env`) to manage secrets safely.
- Gain a solid understanding of web servers such as **Nginx** or **Apache** and how to configure them.
- Learn basic **Linux commands** and file system navigation for server management.

17. Documentation

- Write a clear `README.md` that includes: project overview, setup and installation instructions, usage guidelines, dependencies, and a basic project structure description.
- Document all APIs with request/response examples using Swagger or Postman.
- Follow OpenAPI standards for API documentation.

18. Project Selection Guidelines

- Choose from the suggested MVP list whenever possible.
- If selecting a project outside the provided MVPs, it must have no more than **two user roles**.
- Prioritize learning outcomes over project size or uniqueness.
- If choosing an e-commerce project, it must include **product variants** (at least two).
- Your **Figma design**, **database schema**, and **API documentation** must be consistent and aligned with each other.